



PROJECT DOCUMENTATION

# FitPay — Project Overview

A PT (personal trainer) client management and payment platform, in active real-world use

---

Last updated: 2026-08-20



# Table of Contents

---

- 1. What FitPay Is
- 2. Tech Stack
- 3. Architecture
- 4. Database Schema
- 5. Security
- 6. Features
- 7. Payments (Moolre)
- 8. USSD
- 9. AI (Claude)
- 10. Codebase Map
- 11. Infrastructure & Environments
- 12. Using the App
- 13. Known Gaps

---

## 1. What FitPay Is

---

FitPay replaces the WhatsApp-and-spreadsheet workflow most independent personal trainers run on. A trainer manages their client roster, session bookings, workout programming, and payments in one place. Clients can do everything from a smartphone web app or, since many of a Ghanaian trainer's clients don't carry a data bundle to every gym session, from any basic phone by dialing a USSD code.

It started as a submission to the Moolre Startup Cup 2026, but has moved past that into an ongoing production product with real paying clients.

---

## 2. Tech Stack

---

| Layer     | Choice                 | Why                                                             |
|-----------|------------------------|-----------------------------------------------------------------|
| Framework | Next.js 14, App Router | One codebase for server rendering, client UI, and API routes    |
| Language  | TypeScript (strict)    | End to end type safety, no <code>any</code> in application code |

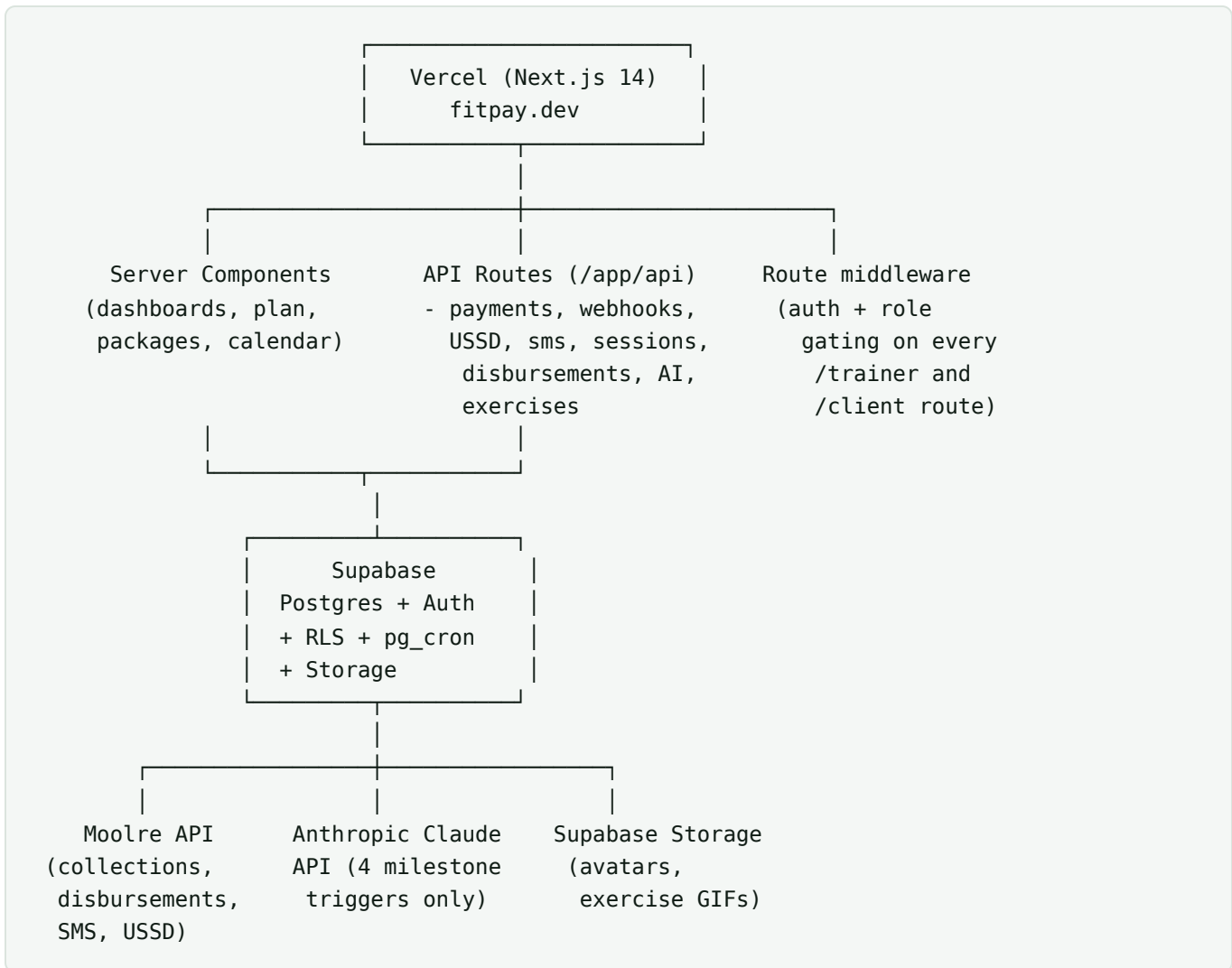
| Layer       | Choice                     | Why                                                                               |
|-------------|----------------------------|-----------------------------------------------------------------------------------|
| Styling     | Tailwind CSS               | Fast iteration, no separate design system to maintain                             |
| Database    | Supabase (PostgreSQL)      | Managed Postgres with auth, Row Level Security, and <code>pg_cron</code> built in |
| Data access | Supabase JS client, no ORM | Hand written SQL migrations, no abstraction overhead                              |
| Payments    | Moolre                     | Ghanaian mobile money collections, disbursements, SMS, and USSD in one provider   |
| AI          | Anthropic Claude API       | Used selectively, not for every workout plan                                      |
| Hosting     | Vercel                     | Native Next.js deployment, connected to a custom domain                           |

Dependencies ( `package.json` ): `next` , `react / react-dom` , `@supabase/supabase-js` + `@supabase/ssr` , `@anthropic-ai/sdk` , `@vercel/functions` , `lucide-react` (icons). No test framework or ORM is currently in the dependency tree.

---

### 3. Architecture

---



**Two Supabase clients** ( `lib/supabase/server.ts` ): - A cookie-bound client using the **anon key**, used only to read the logged-in user's session ( `auth.getUser()` ). RLS applies. - An **admin client** using the **service role key**, used for all actual reads/writes in API routes. RLS is bypassed intentionally, so every API route is responsible for its own authorization checks before touching the admin client.

**Auth and role gating happen in `middleware.ts`**, on every request: unauthenticated users hitting `/client/*` or `/trainer/*` get redirected to `/login`; a client hitting a `/trainer/*` route (or vice versa) gets redirected to their own dashboard.

**Server-to-server calls** (an API route calling another API route, e.g. session-complete triggering the progression engine) are gated by a shared secret header ( `lib/internal.ts` ), checked with a fail-closed default: if the secret env var is unset, internal calls are rejected outright rather than silently allowed.

---

## 4. Database Schema

---

All tables live in Supabase Postgres, created via numbered migrations in `supabase/migrations/`. Every table has Row Level Security enabled — see [Security](#) for the policies.

| Table                          | Purpose                                                                                                                                                                    |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>             | Clients and trainers. <code>role</code> is immutable after signup (see security hardening).                                                                                |
| <code>packages</code>          | Purchasable session bundles (name, session count, price, validity window).                                                                                                 |
| <code>purchases</code>         | A client's package purchase. Carries the Moolre reference as an idempotency key.                                                                                           |
| <code>sessions</code>          | A booked training session, linked to a purchase, with status ( <code>scheduled</code> , <code>completed</code> , <code>cancelled</code> , <code>no_show</code> ).          |
| <code>workout_logs</code>      | What happened in a completed session: exercises performed, difficulty, injury flag, and the generated <code>next_plan</code> .                                             |
| <code>progression_rules</code> | Per-client state for the deterministic rules engine (current phase, sessions in phase, deload cadence).                                                                    |
| <code>disbursements</code>     | Trainer withdrawals and client refunds paid out via Moolre.                                                                                                                |
| <code>refund_requests</code>   | A client's refund request and the trainer's approve/reject decision.                                                                                                       |
| <code>medical_history</code>   | Client medical clearance and injury history, collected at signup.                                                                                                          |
| <code>ussd_sessions</code>     | Short-lived state for an in-progress USSD session (service role only).                                                                                                     |
| <code>exercises</code>         | Read-only reference library: 1,324 exercises with body part, equipment, target muscle, and a technique GIF (seeded from a third-party dataset, see <a href="#">§6.9</a> ). |

The `exercise_entry` shape ( { `name`, `sets`, `reps`, `weight_kg`, `difficulty`, `notes?`, `exercise_id?`, `gif_url?` } ) is enforced at the application layer everywhere workouts are stored, never as free-form text.

---

## 5. Security

---

### 5.1 Row Level Security

Every table has RLS enabled with explicit policies, roughly following this pattern:

- `users` : a user can read/update only their own row.

- `purchases` , `sessions` , `workout_logs` , `progression_rules` , `refund_requests` , `medical_history` : a client sees only their own rows; a trainer sees only the rows belonging to their own clients (multi-trainer isolation, migration `008_multi_trainer.sql` ).
- `packages` , `exercises` : readable by any authenticated user, since they're shared reference data.
- `disbursements` : trainer only.
- `ussd_sessions` : no client access at all — service role (server) only.

## 5.2 Privilege escalation fix (migration 009)

Originally, the self-signup `INSERT` policy on `users` let anyone insert a row with `role = 'trainer'` , and the `UPDATE` policy let a client flip their own role to `trainer` — which would have granted full access to every client's PII, medical records, refund approvals, and withdrawal of platform funds. This was closed in two steps:

1. The insert policy now hard-requires `role = 'client'` ; a trainer account can only be provisioned out-of-band (direct `INSERT` via service role).
2. A `BEFORE UPDATE` trigger ( `prevent_role_change` ) makes `role` immutable after creation, so no update path can change it, regardless of how the request is shaped.

## 5.3 Internal route protection

Routes that should only ever be called server-to-server (SMS sending, the AI triggers, the progression engine) check `x-internal-secret` against `INTERNAL_API_SECRET` ( `lib/internal.ts` ). If the secret env var is missing, requests are rejected — the check fails closed, not open.

## 5.4 Webhook idempotency

`app/api/webhooks/moolre/route.ts` follows a strict pattern:

1. Verify the shared webhook secret; fail closed if unset.
2. Look up the purchase by `moolre_ref` .
3. If it's already `active` , return `200` immediately without reprocessing.
4. Otherwise activate it (conditioned on `status = 'pending'` to avoid a race).
5. Always return HTTP `200` , even for ignored/duplicate/invalid payloads — Moolre retries on any non-200, and retried duplicates must be safe no-ops.

## 5.5 Secrets and environment separation

- Only `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` are exposed to the browser. Every other credential ( `SUPABASE_SERVICE_ROLE_KEY` , `MOOLRE_API_KEY` , `ANTHROPIC_API_KEY` , `MOOLRE_WEBHOOK_SECRET` , `INTERNAL_API_SECRET` ) is server-only.
- All Moolre API calls happen from API routes, never from a client component.

- Sandbox and live Moolre credentials are separate; sandbox is used everywhere except the live production environment.

## 5.6 HTTP security headers ( `next.config.mjs` )

Applied globally: `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin` , `Permissions-Policy: camera=(), microphone=(), geolocation=()` , and HSTS ( `Strict-Transport-Security` , 2-year max-age, includes subdomains, preload).

## 5.7 Auth flows

- Standard email/password via Supabase Auth.
- Forgot-password uses Supabase's PKCE recovery flow: `resetPasswordForEmail` → `/auth/callback` exchanges the code for a session → `/reset-password` lets the user set a new password → session is force-signed-out and the user is redirected to log in fresh. The flow always shows a generic success message regardless of whether the email exists, to avoid leaking which emails are registered.
- Account deletion is user-initiated ( `DeleteAccountButton` ), requires typing `DELETE` to confirm, and is irreversible.

## 5.8 Known limitation: pooled funds

There is currently no per-trainer fund segregation at the payment-provider level — see [§13 Known Gaps](#).

---

# 6. Features

---

## 6.1 Trainer dashboard

An operational home screen: today's and upcoming sessions, roster health, quick links to log a session, view earnings, and review pending refund requests.

## 6.2 Client dashboard

A mobile-first home screen: next session, sessions remaining, a link into their plan and progress, and a profile with avatar.



## 6.3 Session booking

Clients pick an open slot from a fixed set of daily times ( `lib/slots.ts` — 6am, 8am, noon, 4pm, 6pm, all computed in UTC since Ghana has no DST, so slot math is stable regardless of server timezone) via a calendar UI, or book the same way over USSD. Every slot is single-trainer, single-booking (no double-booking a time). Confirmations go out by SMS automatically.

## 6.4 Session logging and the progression engine

After each session, the trainer logs what was actually done: exercises, sets/reps/weight, overall difficulty, and an optional injury flag. A deterministic rules engine ( `lib/engine/progression.ts` ) then generates the client's *next* session automatically:

- Every 4th session (configurable via `deload_every_n` ) becomes a deload — all weights cut 40%, same movements.
- Otherwise, each exercise adjusts based on how hard it was: easy → +2.5kg or +2 reps (goal-dependent), moderate → unchanged, hard → -1 rep (weight held).
- Muscle groups rotate so the same primary group never repeats on consecutive sessions.
- Exercise selection routes by the client's goal (weight loss: higher reps + compound movements + a cardio finisher; strength: low reps, heavy compounds; endurance: circuit style; general: balanced).

This engine runs on every session, for free, with no AI call involved — see §9 AI for where Claude is used.

## 6.5 Client plan view

Clients see their next session's exercises, with technique GIFs where available, and a banner when a plan has been adapted for a reported injury.

## 6.6 Packages and payments

Clients buy session packages (a name, a session count, a price, a validity window) via Moolre mobile money, in-app or by USSD. Purchases activate the moment the payment webhook confirms.

## 6.7 Refunds

A client can request a refund on an active purchase; the trainer approves or rejects it. Approved refunds calculate a live pro-rated amount based on sessions already used, and pay out via Moolre disbursement to the client's phone.

## 6.8 Trainer earnings and withdrawals

The trainer's dashboard shows their available balance (an internal ledger: sum of their clients' purchases minus what's already been disbursed to them) and lets them withdraw to mobile money.

## 6.9 Exercise library with technique GIFs

A searchable library of 1,324 exercises (name, body part, equipment, target muscle, and an animated technique GIF), seeded from a third-party open dataset ( `hasaneyldrm/exercises-dataset` , MIT-licensed data, Gym visual-licensed media). Used in the trainer's session-log form as an autocomplete picker (free typing still works; picking a result attaches the GIF), and the GIF then travels with that `exercise_entry` wherever it's used, including forward into future sessions via the progression engine, and is shown to the client on their plan page. Gym visual's required attribution is shown next to every GIF.

## 6.10 Medical history and clearance

Clients complete a medical history and clearance step during signup (injuries, conditions, consent). Trainers can review and mark it reviewed.

## 6.11 Multi-trainer support

The schema and RLS policies support more than one trainer on the platform, each seeing only their own client roster, sessions, and disbursements — though trainer accounts are provisioned out-of-band rather than via self-signup (see §5.2).

## 6.12 Calendar export

Clients can add a booked session to their own calendar via a generated Google Calendar link or a downloadable `.ics` file (works with Apple Calendar and Outlook too), no OAuth required ( `lib/calendar.ts` ).

## 6.13 PWA install

FitPay is installable as a Progressive Web App ( `public/manifest.json` , iOS splash screens, app icons, an in-app branded launch splash screen), so clients get an app-like experience without an app store.

## 6.14 Scheduled SMS check-ins

A monthly `pg_cron` job triggers a personalised SMS check-in to every active client (see §9.4).

---

## 7. Payments (Moolre)

---

All Moolre integration is server-side only ( `lib/moolre.ts` and the routes under `app/api/payments` , `app/api/disbursements` , `app/api/sms` , `app/api/usssd` , `app/api/webhooks/moolre` ).

- **Collections:** `POST /payment/initiate` to start a client's package purchase, `POST /payment/status` to poll, and a webhook that Moolre calls on status change.
- **Disbursements:** `POST /transfer/initiate` / `POST /transfer/status` , used for trainer withdrawals and client refunds.
- **SMS:** `POST /sms/send` , used for booking confirmations, payment links, and AI-drafted check-ins/progress reports.
- **USSD:** Moolre posts to `app/api/usssd/callback` on every keypress; the endpoint must respond within 5 seconds or the session dies.
- Every response follows `{ status, code, message, data, go }` — success is checked via `status === 1` , never HTTP status alone.
- A single pooled `M00LRE_ACCOUNT_NUMBER` is used for all trainers currently (see §13).

### GSM transport safety

USSD and SMS only support the GSM 7-bit alphabet — a real bug surfaced this: the cedi sign (¢) and typographic middle dot (·) were being sent as raw UTF-8 and arriving on real phones as mojibake (garbled bytes reinterpreted as Latin-1). This is now handled at three layers: plain ASCII is used directly in USSD/SMS message text (GHS instead of ¢, a hyphen instead of ·), Claude's SMS-generation prompts are explicitly instructed to stay ASCII-only, and a `toGsmSafe()` sanitizer runs as a last line of defense on every outbound SMS body and USSD reply. Web-app UI text is unaffected — browsers render UTF-8 fine, so ¢ still displays correctly in the app itself.

---

## 8. USSD

---

The live dial code is `*919*4012#` (Moolre-provisioned, no telco setup needed). State for an in-progress session lives in `usssd_sessions` (service-role only, short TTL).

```
START
└─ main_menu
    │ 1 → book_session → show_slots → confirm_slot → BOOK → END (+ SMS confirmation)
    │ 2 → view_plan → show next session from the rules engine → END
    │ 3 → pay_sessions → show_packages → select_package → initiate payment → END
    │ 4 → session_balance → show_sessions_left → END
    └─ 0 → EXIT
```

Every step handles: a valid input, an invalid input (re-shows the menu with an error instead of failing), `0` to go back a level, and session timeout with a clean exit message. Responses use the `CON` prefix to keep a session open and `END` to close it.

---

## 9. AI (Claude)

Claude is deliberately **not** used for routine workout planning — that's the deterministic rules engine's job, for cost, reliability, and scale reasons. Claude fires on exactly four milestone triggers, each its own route under `app/api/ai/`:

### 9.1 New client onboarding ( `/api/ai/onboard` )

Fires once per client at signup. Given goal, fitness level, injuries, and available days, generates an initial 4-session starter plan as structured JSON ( `max_tokens: 800` ).

### 9.2 Progress report ( `/api/ai/report` )

Fires every 4th completed session. Given the last 4 workout logs and the client's goal, generates a short narrative (not a plan) on what improved and what to focus on, delivered by SMS ( `max_tokens: 500` ).

### 9.3 Injury adaptation ( `/api/ai/adapt` )

Fires when a workout log is flagged with an injury. Given the injury notes and recent session history, restructures the next 2 sessions to avoid the injured area.

### 9.4 Monthly check-in ( `/api/ai/checkin` )

Fires via `pg_cron` on the 1st of each month for every active client. Drafts a short, personalised SMS check-in (max 160 characters) based on goal and recent activity.

Every SMS-facing prompt includes an explicit ASCII-only instruction (see [GSM transport safety](#)). If the Claude API is unavailable, the calling route is expected to queue the task rather than break the user-facing flow.



## 10. Codebase Map

---

```
app/
├─ (auth)/          login, signup, forgot-password, reset-password
├─ (client)/client/  dashboard, book, calendar, packages, plan, progress, profile
├─ (trainer)/trainer/ dashboard, calendar, clients, sessions, log, earnings
├─ onboarding/       medical-history intake
├─ auth/callback/    Supabase PKCE recovery code exchange
├─ api/
│   ├─ ai/           onboard, report, adapt, checkin
│   ├─ calendar/     per-session .ics/Google Calendar link
│   ├─ clients/       medical-review
│   ├─ disbursements/ withdraw, request-refund, approve-refund, reject-refund, refund
│   ├─ engine/        progress (rules engine trigger)
│   ├─ exercises/     search (exercise library autocomplete)
│   ├─ payments/      initiate, status
│   ├─ sessions/      book, cancel, complete, availability
│   ├─ sms/           send
│   ├─ trainers/      trainer lookup
│   ├─ ussd/          callback
│   └─ webhooks/moolre/ payment webhook (idempotent)
├─ account/         delete
├─ page.tsx         public landing page
└─ middleware.ts     auth + role gating on every request

lib/
├─ moolre.ts         Moolre API client (server-only) + GSM-safe sanitizer
├─ ai/claude.ts      Claude API wrapper (server-only)
├─ engine/progression.ts deterministic workout rules engine
├─ sessions/book.ts  shared booking logic
├─ slots.ts          fixed daily slot definitions (UTC-stable)
├─ calendar.ts       Google Calendar link + .ics generation
├─ internal.ts       fail-closed internal service-to-service auth
├─ appleSplash.ts    iOS PWA splash screen config
├─ supabase/
│   ├─ client.ts     browser client (anon key)
│   └─ server.ts     cookie client (auth only) + admin client (service role)

components/
├─ ExercisePicker.tsx    autocomplete search + GIF preview for logging exercises
├─ GoalSelector.tsx      multi-select fitness goal picker
├─ MedicalHistoryFields.tsx
├─ AddToCalendar.tsx
├─ CancelSessionButton.tsx
├─ DeleteAccountButton.tsx
├─ SplashScreen.tsx
└─ skeletons.tsx        route-level loading states
```

```

scripts/
└─ seed-exercises.mjs      one-off importer for the exercise library dataset

supabase/
├─ migrations/            001 through 014, numbered and sequential
└─ email-templates/      branded reset-password email (matches app palette)

types/index.ts            shared TypeScript types (User, Session, ExerciseEntry, etc.)

```

## 11. Infrastructure & Environments

| Environment | Supabase Project | Moolre  | Purpose                                |
|-------------|------------------|---------|----------------------------------------|
| development | fitpay-dev       | Sandbox | Daily building                         |
| demo        | fitpay-demo      | Sandbox | Sales demos, stakeholder walkthroughs  |
| production  | fitpay-prod      | Live    | Real trainer, real clients, live money |

- **Domain:** `fitpay.dev`, a paid custom domain connected via Vercel Domains (not the default `.vercel.app`). `NEXT_PUBLIC_APP_URL`, the Supabase Auth redirect URLs, and the `monthly-client-checkin` `pg_cron` job all point at it.
- **Hosting:** Vercel, auto-deploying from the repository's `main` branch. Feature branches get their own Vercel preview deployments.
- **Scheduling:** `pg_cron`, running inside Supabase's Postgres instance, not Vercel cron (which has meaningful limitations on the free tier). Secrets read by cron jobs (e.g. `INTERNAL_API_SECRET`) are stored in Supabase Vault rather than database-level config.
- **Storage:** Supabase Storage — an `avatars` bucket (public read, user-scoped write) and an `exercise-media` bucket (public, seeded by the exercise-library import script).
- **Environment variables:** ```` # Public (client-safe) NEXT_PUBLIC_SUPABASE_URL= NEXT_PUBLIC_SUPABASE_ANON_KEY= NEXT_PUBLIC_APP_URL=https://fitpay.dev`

```

# Server-only SUPABASE_SERVICE_ROLE_KEY= MOOLRE_API_USER= MOOLRE_API_KEY=
MOOLRE_SANDBOX_API_USER= MOOLRE_SANDBOX_API_KEY= MOOLRE_WEBHOOK_SECRET=
MOOLRE_ACCOUNT_NUMBER= ANTHROPIC_API_KEY= INTERNAL_API_SECRET= USE_SANDBOX=true
# flip to false for live production ```

```

## 12. Using the App

---

### As a client

1. Sign up, complete medical history and clearance, select fitness goals.
2. Receive an AI-generated starter plan once the trainer's roster picks it up.
3. Book sessions from the calendar (web) or by dialing `*919*4012#` (any phone).
4. Buy session packages via mobile money, in-app or by USSD.
5. After each session, view the next session's plan, with technique GIFs, on the Plan page.
6. Track progress, request refunds on unused sessions, manage your profile, or delete your account entirely if you choose to leave.

### As the trainer

1. Provisioned directly (not via self-signup, see §5.2).
2. See today's and upcoming sessions on the dashboard.
3. Log each completed session: search the exercise library or type free text, record sets/reps/weight/difficulty, flag injuries when relevant.
4. The rules engine (or Claude, for injury cases) generates the client's next session automatically.
5. Review and act on refund requests.
6. Track and withdraw earnings.

### Seeding the exercise library (one-time, per environment)

```
git clone https://github.com/hasaneyldrm/exercises-dataset /tmp/exercises-dataset
NEXT_PUBLIC_SUPABASE_URL=... SUPABASE_SERVICE_ROLE_KEY=... \
node scripts/seed-exercises.mjs --source /tmp/exercises-dataset
```

Idempotent — safe to re-run.

---

## 13. Known Gaps

---

Documented in `CLAUDE.md` as intentional, acknowledged limitations, not oversights:

1. **No 1% commission mechanism.** The landing page states FitPay takes 1% per transaction, but there is no client-count gating, fee calculation, ledger entry, or destination account for platform revenue anywhere in the codebase yet.



2. **Single pooled Moolre account, no real fund segregation.** All collections and disbursements for every trainer flow through one `M00LRE_ACCOUNT_NUMBER` . Per-trainer balances exist only as an internal ledger (purchases minus disbursements per `trainer_id` ) — not as provider-level separated funds. This matches Moolre's standard API and the current single-trainer reality, but at multi-trainer scale it means shared exposure to freezes, disputes, or float shortages, and trust concentration on the app's own bookkeeping rather than any provider-level guarantee.

Both are flagged here for whenever fee monetization or multi-trainer scaling becomes a priority.